

The Conjugate Gradient Method

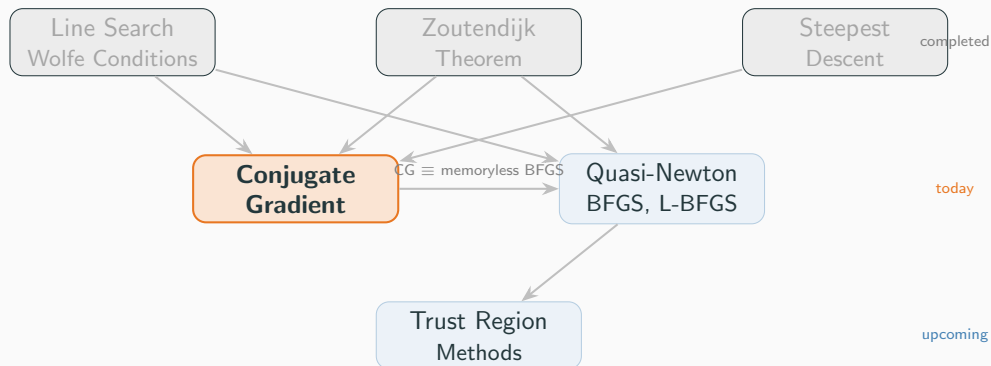
It's Just Gram–Schmidt (With a Twist)

Robert Hildebrand

ISE 5405/5406 — Virginia Tech

Where We Are

How This Fits Into the Course



Today's ingredients: Steepest descent (what goes wrong), Wolfe conditions (how we do line search), and Zoutendijk (how we prove convergence).

Motivation: Why Steepest Descent Fails

The Problem

We want to minimize a smooth function $f : \mathbb{R}^n \rightarrow \mathbb{R}$.

Start simple: Consider the quadratic

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top \mathbf{A} \mathbf{x} - \mathbf{b}^\top \mathbf{x},$$

where $\mathbf{A} \in \mathbb{R}^{n \times n}$ is symmetric positive definite.

- Gradient: $\nabla f(\mathbf{x}) = \mathbf{A} \mathbf{x} - \mathbf{b}$
- Unique minimizer: $\mathbf{x}^* = \mathbf{A}^{-1} \mathbf{b}$
- Residual: $\mathbf{r}_k = \mathbf{b} - \mathbf{A} \mathbf{x}_k = -\nabla f(\mathbf{x}_k)$

Goal: Find $\mathbf{x}^*$ without computing $\mathbf{A}^{-1}$. **Notation:** We write

$\nabla f(\mathbf{x}_k) = \nabla f(\mathbf{x}_k) = \mathbf{A} \mathbf{x}_k - \mathbf{b}$ for the gradient, and $\mathbf{r}_k = -\nabla f(\mathbf{x}_k)$ for the residual.

Steepest Descent on a Quadratic

Steepest descent: At each step, move in the direction $-\nabla f(\mathbf{x}_k)$ with exact line search:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k(-\nabla f(\mathbf{x}_k)), \quad \alpha_k = \frac{\nabla f(\mathbf{x}_k)^\top \nabla f(\mathbf{x}_k)}{\nabla f(\mathbf{x}_k)^\top \mathbf{A} \nabla f(\mathbf{x}_k)}.$$

Key property of exact line search:

$$\nabla f(\mathbf{x}_{k+1})^\top \nabla f(\mathbf{x}_k) = 0.$$

Each gradient is orthogonal to the previous one.

Sounds good. What goes wrong?

Exercise: Gradient and Step Size

Consider $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^\top \mathbf{A}\mathbf{x} - \mathbf{b}^\top \mathbf{x}$ with $\mathbf{A} = \begin{pmatrix} 4 & 0 \\ 0 & 1 \end{pmatrix}$, $\mathbf{b} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$, $\mathbf{x}_0 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}$.

1. Compute $\nabla f(\mathbf{x}_0)$.
2. Steepest descent takes $\mathbf{d}_0 = -\nabla f(\mathbf{x}_0)$. To find the step size, substitute $\mathbf{x}_0 + \alpha \mathbf{d}_0$ into f and write $\phi(\alpha) = f(\mathbf{x}_0 + \alpha \mathbf{d}_0)$ as a polynomial in α .
3. Minimize $\phi(\alpha)$ to find the exact line search step size α_0 . Compute $\mathbf{x}_1$.
4. Compute $\nabla f(\mathbf{x}_1)$. Verify that $\nabla f(\mathbf{x}_1)^\top \mathbf{d}_0 = 0$.
5. **Bonus:** Now take $\mathbf{d}_0 = (1, -1)^\top$ instead of the negative gradient. Find α_0 by the same method. Is $\nabla f(\mathbf{x}_1)^\top \mathbf{d}_0 = 0$ still true?

Deriving the Step Size: Exact Line Search on a Quadratic

Given a search direction $\mathbf{d}_k$ (not necessarily $-\nabla f(\mathbf{x}_k)$), we want the α that minimizes f along the ray $\mathbf{x}_k + \alpha \mathbf{d}_k$.

Substitute into $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^\top \mathbf{A}\mathbf{x} - \mathbf{b}^\top \mathbf{x}$:

$$\begin{aligned}\phi(\alpha) &= f(\mathbf{x}_k + \alpha \mathbf{d}_k) \\ &= \underbrace{f(\mathbf{x}_k)}_{\text{constant}} + \underbrace{\alpha (\mathbf{A}\mathbf{x}_k - \mathbf{b})^\top}_{= \nabla f(\mathbf{x}_k)^\top} \mathbf{d}_k + \frac{1}{2}\alpha^2 \mathbf{d}_k^\top \mathbf{A} \mathbf{d}_k\end{aligned}$$

This is a **quadratic in α** —a parabola opening upward (since $\mathbf{A} \succ 0 \Rightarrow \mathbf{d}_k^\top \mathbf{A} \mathbf{d}_k > 0$).

Set $\phi'(\alpha) = 0$:

$$\phi'(\alpha) = \nabla f(\mathbf{x}_k)^\top \mathbf{d}_k + \alpha \mathbf{d}_k^\top \mathbf{A} \mathbf{d}_k = 0$$

$$\boxed{\alpha_k = \frac{-\nabla f(\mathbf{x}_k)^\top \mathbf{d}_k}{\mathbf{d}_k^\top \mathbf{A} \mathbf{d}_k}}$$

This is the **exact line search** step size for *any* direction $\mathbf{d}_k$ on a quadratic.

Special Cases and the Orthogonality Property

For steepest descent: $\mathbf{d}_k = -\nabla f(\mathbf{x}_k)$, so:

$$\alpha_k = \frac{-\nabla f(\mathbf{x}_k)^\top (-\nabla f(\mathbf{x}_k))}{(-\nabla f(\mathbf{x}_k))^\top \mathbf{A} (-\nabla f(\mathbf{x}_k))} = \frac{\|\nabla f(\mathbf{x}_k)\|^2}{\nabla f(\mathbf{x}_k)^\top \mathbf{A} \nabla f(\mathbf{x}_k)}.$$

For CG (or any direction): use the general formula $\alpha_k = \frac{-\nabla f(\mathbf{x}_k)^\top \mathbf{d}_k}{\mathbf{d}_k^\top \mathbf{A} \mathbf{d}_k}$.

In residual notation ($\mathbf{r}_k = -\nabla f(\mathbf{x}_k)$): $\alpha_k = \frac{\mathbf{r}_k^\top \mathbf{d}_k}{\mathbf{d}_k^\top \mathbf{A} \mathbf{d}_k}$.

Why the next gradient is orthogonal to $\mathbf{d}_k$:

The gradient at $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k$ is:

$$\nabla f(\mathbf{x}_{k+1}) = \nabla f(\mathbf{x}_k) + \alpha_k \mathbf{A} \mathbf{d}_k.$$

So:

$$\nabla f(\mathbf{x}_{k+1})^\top \mathbf{d}_k = \nabla f(\mathbf{x}_k)^\top \mathbf{d}_k + \alpha_k \mathbf{d}_k^\top \mathbf{A} \mathbf{d}_k = \nabla f(\mathbf{x}_k)^\top \mathbf{d}_k + \frac{-\nabla f(\mathbf{x}_k)^\top \mathbf{d}_k}{\mathbf{d}_k^\top \mathbf{A} \mathbf{d}_k} \cdot \mathbf{d}_k^\top \mathbf{A} \mathbf{d}_k = 0.$$

Exact line search $\Rightarrow \nabla f(\mathbf{x}_{k+1}) \perp \mathbf{d}_k$. Always. Regardless of how $\mathbf{d}_k$ was chosen.

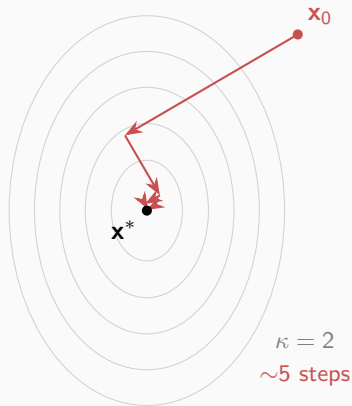
Example: A Well-Conditioned Problem ($\kappa = 2$)

$$f(x_1, x_2) = x_1^2 + \frac{1}{2}x_2^2$$

$$\mathbf{A} = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}, \quad \kappa = 2, \quad \mathbf{x}_0 = (3, 3.5)^\top.$$

Steepest descent: mild zigzag, converges in a few steps.

Each step is $\perp$ to the last (exact line search). Visible zigzag, but mild.



Example: An Ill-Conditioned Problem ($\kappa = 50$)

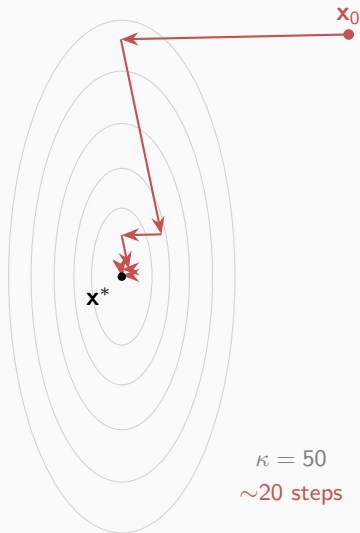
$$f(x_1, x_2) = 25x_1^2 + \frac{1}{2}x_2^2$$

$$\mathbf{A} = \begin{pmatrix} 50 & 0 \\ 0 & 1 \end{pmatrix}, \quad \kappa = 50, \quad \mathbf{x}_0 = (1.5, 5)^\top.$$

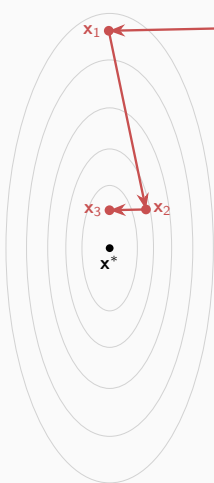
Gradient at $\mathbf{x}_0$: $(75, 5)^\top$ —almost entirely in x_1 . So SD goes nearly horizontal.

Then line search overshoots in x_1 , next gradient is nearly vertical, and the process repeats.

The zigzag is real: ~ 20 steps visible here.



Notice: Steps Are Perpendicular to the Contours



Step 0 $\rightarrow$ 1: $\nabla f = (75, 5)^T$
Almost horizontal left
 $\perp$ to tall contours $\checkmark$

Step 1 $\rightarrow$ 2: $\nabla f \approx (0, 4.9)^T$
Almost vertical down
 $\perp$ to wide part $\checkmark$

Each step *is* orthogonal to
the contours—that's the
definition of the gradient!
But it keeps alternating.

CG on the Same Problem: Two Steps

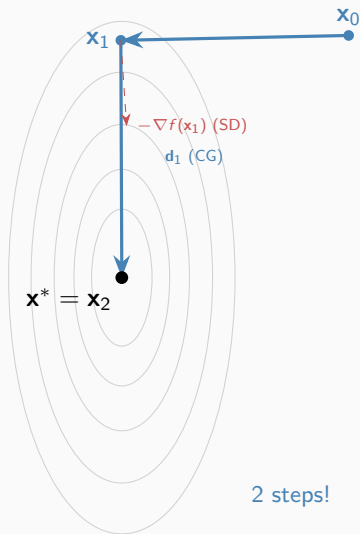
Same problem: $\kappa = 50$, $\mathbf{x}_0 = (1.5, 5)^\top$.

CG builds **A**-conjugate directions. First step is identical to SD.

Second step: SD would go nearly horizontal ($-\nabla f(\mathbf{x}_1)$ is mostly in x_1). CG instead tilts the direction to head straight for $\mathbf{x}^*$.

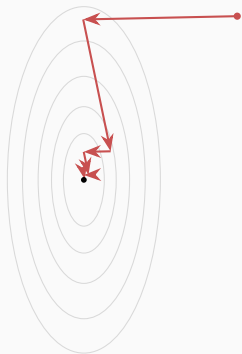
CG converges in exactly 2 steps (since $n = 2$).

2 steps vs. ~ 20 steps.



Side by Side: SD vs. CG

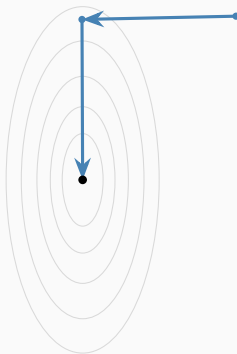
Steepest Descent



~20 steps



Conjugate Gradient



2 steps (exact)

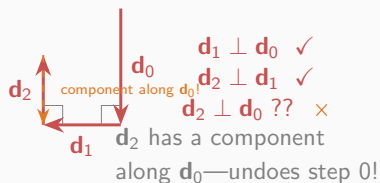
The Diagnosis

Why does steepest descent zigzag?

Exact line search along $\mathbf{d}_k = -\nabla f(\mathbf{x}_k)$ gives $\nabla f(\mathbf{x}_{k+1})^\top \mathbf{d}_k = 0$.

But it does *not* guarantee $\nabla f(\mathbf{x}_{k+1})^\top \mathbf{d}_j = 0$ for $j < k$.

Each new step can *undo* progress made in earlier directions.



The Design Question

Can we choose search directions $\mathbf{d}_0, \mathbf{d}_1, \dots$ so that the progress along each direction is **permanent**—never revisited, never undone?

Yes. The key insight:

Orthogonality in the *standard* inner product $\Rightarrow$ zigzag.

Orthogonality in the **A**-inner product $\langle \mathbf{u}, \mathbf{v} \rangle_A = \mathbf{u}^\top \mathbf{A} \mathbf{v} \Rightarrow$ permanent progress.

Conjugate Directions: The Idea

The Conjugacy Condition

Theorem

If $\{\mathbf{d}_0, \dots, \mathbf{d}_{n-1}\}$ are $\mathbf{A}$ -conjugate, i.e.,

$$\mathbf{d}_i^\top \mathbf{A} \mathbf{d}_j = 0 \quad \text{for } i \neq j,$$

then exact line search along these directions **terminates in at most n steps**:

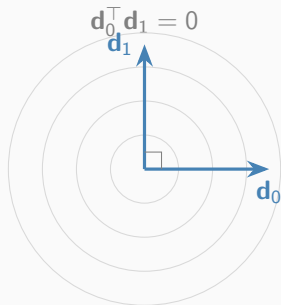
$$\mathbf{x}_n = \mathbf{x}^* = \mathbf{A}^{-1}\mathbf{b}.$$

Why? Write $\mathbf{x}^* - \mathbf{x}_0 = \sum_{i=0}^{n-1} \alpha_i \mathbf{d}_i$ (the $\mathbf{d}_i$ are a basis). $\mathbf{A}$ -conjugacy *decouples* the coordinates: each exact line search recovers α_i independently of the others.

Analogy: Diagonalizing the problem without computing eigenvectors.

Geometric Intuition: Standard vs. A-Orthogonality

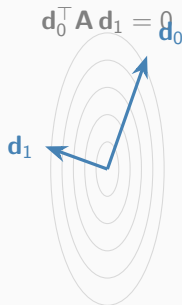
Standard Orthogonality



For $\mathbf{A} = \mathbf{I}$: circles
Standard $\perp = \mathbf{A}\text{-}\perp$

different "orthogonal" means ---

A-Conjugacy



For general $\mathbf{A}$: ellipses
Visually not $\perp$, but $\mathbf{A}\text{-}\perp$!

Exercise: Discovering Why Steps Don't Interfere

Use $\mathbf{A} = \begin{pmatrix} 4 & 0 \\ 0 & 1 \end{pmatrix}$, $\mathbf{b} = \mathbf{0}$, $\mathbf{x}_0 = (2, 4)^\top$.

Take two $\mathbf{A}$ -conjugate directions: $\mathbf{d}_0 = (1, 0)^\top$, $\mathbf{d}_1 = (0, 1)^\top$. (Verify: $\mathbf{d}_0^\top \mathbf{A} \mathbf{d}_1 = ?$)

1. Compute the exact line search step along $\mathbf{d}_0$:

$$\alpha_0 = \frac{-\nabla f(\mathbf{x}_0)^\top \mathbf{d}_0}{\mathbf{d}_0^\top \mathbf{A} \mathbf{d}_0}, \quad \mathbf{x}_1 = \mathbf{x}_0 + \alpha_0 \mathbf{d}_0.$$

2. Compute $\nabla f(\mathbf{x}_1) = \nabla f(\mathbf{x}_1)$ and verify that $\nabla f(\mathbf{x}_1)^\top \mathbf{d}_0 = 0$.
3. Now take the step along $\mathbf{d}_1$: compute α_1 and $\mathbf{x}_2 = \mathbf{x}_1 + \alpha_1 \mathbf{d}_1$.
4. Compute $\nabla f(\mathbf{x}_2) = \nabla f(\mathbf{x}_2)$. Is $\nabla f(\mathbf{x}_2)^\top \mathbf{d}_0 = 0$? Is $\nabla f(\mathbf{x}_2)^\top \mathbf{d}_1 = 0$?
5. **The key question:** Taking the step along $\mathbf{d}_1$ didn't undo the progress along $\mathbf{d}_0$. Why not? What property of $\mathbf{d}_0$ and $\mathbf{d}_1$ made this work?
Hint: Use $\nabla f(\mathbf{x}_2) = \nabla f(\mathbf{x}_1) + \alpha_1 \mathbf{A} \mathbf{d}_1$ and the fact that $\nabla f(\mathbf{x}_1)^\top \mathbf{d}_0 = 0$.

Proof: Why Steps Don't Interfere

After $k + 1$ steps: $\mathbf{x}_{k+1} = \mathbf{x}_0 + \sum_{i=0}^k \alpha_i \mathbf{d}_i$, so $\nabla f(\mathbf{x}_{k+1}) = \nabla f(\mathbf{x}_0) + \sum_{i=0}^k \alpha_i \mathbf{A} \mathbf{d}_i$.

For any $j \leq k$:

$$\nabla f(\mathbf{x}_{k+1})^\top \mathbf{d}_j = \nabla f(\mathbf{x}_0)^\top \mathbf{d}_j + \sum_{i=0}^k \alpha_i \underbrace{\mathbf{d}_j^\top \mathbf{A} \mathbf{d}_i}_{=0 \text{ if } i \neq j} = \nabla f(\mathbf{x}_0)^\top \mathbf{d}_j + \alpha_j \mathbf{d}_j^\top \mathbf{A} \mathbf{d}_j.$$

Since $\alpha_j = -\nabla f(\mathbf{x}_j)^\top \mathbf{d}_j / (\mathbf{d}_j^\top \mathbf{A} \mathbf{d}_j)$ and by a similar telescoping argument $\nabla f(\mathbf{x}_j)^\top \mathbf{d}_j = \nabla f(\mathbf{x}_0)^\top \mathbf{d}_j$:

$$\nabla f(\mathbf{x}_{k+1})^\top \mathbf{d}_j = \nabla f(\mathbf{x}_0)^\top \mathbf{d}_j - \nabla f(\mathbf{x}_0)^\top \mathbf{d}_j = 0. \quad \square$$

Permanent progress: after step k , the gradient has no component along $\mathbf{d}_0, \dots, \mathbf{d}_k$.

Exercise: From Gradients to Conjugate Directions

Suppose we've just run two SD steps on $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^\top \mathbf{A}\mathbf{x}$ and have $-\nabla f(\mathbf{x}_0)$ and $-\nabla f(\mathbf{x}_1)$. These are *not* $\mathbf{A}$ -conjugate in general. We want to build $\mathbf{d}_0, \mathbf{d}_1$ that *are*.

Use $\mathbf{A} = \begin{pmatrix} 6 & 2 \\ 2 & 4 \end{pmatrix}$, $\mathbf{b} = \mathbf{0}$, $\mathbf{x}_0 = (1, 1)^\top$.

1. Set $\mathbf{d}_0 = -\nabla f(\mathbf{x}_0) = -\mathbf{A}\mathbf{x}_0$. Compute $\mathbf{d}_0$.
2. Run one CG step: compute α_0 and $\mathbf{x}_1$, then $\nabla f(\mathbf{x}_1) = \mathbf{A}\mathbf{x}_1$.
3. We want $\mathbf{d}_1 = -\nabla f(\mathbf{x}_1) + \beta \mathbf{d}_0$ to satisfy $\mathbf{d}_1^\top \mathbf{A} \mathbf{d}_0 = 0$.
Expand this condition and solve for β :

$$(-\nabla f(\mathbf{x}_1) + \beta \mathbf{d}_0)^\top \mathbf{A} \mathbf{d}_0 = 0 \implies \beta = ?$$

4. Compute $\mathbf{d}_1$. Verify: is $\mathbf{d}_1^\top \mathbf{A} \mathbf{d}_0 = 0$?
5. Compare your β with $\beta^{\text{FR}} = \|\nabla f(\mathbf{x}_1)\|^2 / \|\nabla f(\mathbf{x}_0)\|^2$. Are they the same?
This is the Gram–Schmidt coefficient in the $\mathbf{A}$ -inner product, which turns out to equal the simple FR formula. The next slides explain why.

Building Conjugate Directions: Gram–Schmidt

The Natural Approach: A-Gram–Schmidt

We have the negative gradients $-\nabla f(\mathbf{x}_0), -\nabla f(\mathbf{x}_1), -\nabla f(\mathbf{x}_2), \dots$ as raw material.

Idea: Gram–Schmidt orthogonalization with the $\mathbf{A}$ -inner product $\langle \mathbf{u}, \mathbf{v} \rangle_A = \mathbf{u}^\top \mathbf{A} \mathbf{v}$:

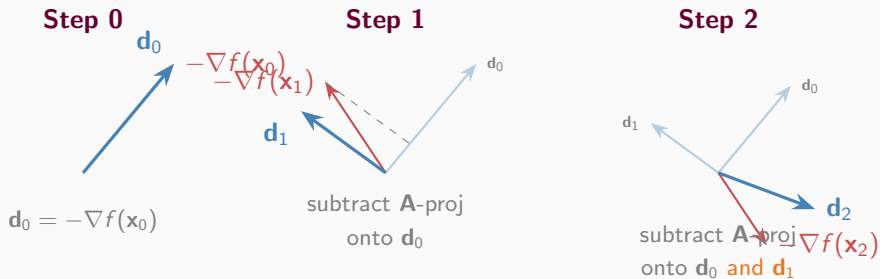
$$\mathbf{d}_0 = -\nabla f(\mathbf{x}_0)$$

$$\mathbf{d}_k = -\nabla f(\mathbf{x}_k) + \sum_{j=0}^{k-1} \beta_{kj} \mathbf{d}_j, \quad \beta_{kj} = \frac{\nabla f(\mathbf{x}_k)^\top \mathbf{A} \mathbf{d}_j}{\mathbf{d}_j^\top \mathbf{A} \mathbf{d}_j}.$$

Problem: At step k , this requires k inner products and storing all previous directions.

Total work: $O(n^2)$. No better than factoring $\mathbf{A}$.

Visualizing Full Gram–Schmidt



At step k : project out components along *all* k previous directions.

$O(kn)$ work per step, $O(n^2)$ total. Too expensive!

The CG Miracle: Short Recurrence

Theorem (The CG Miracle)

With exact line search on the quadratic, full **A**-Gram-Schmidt collapses to:

$$\mathbf{d}_k = -\nabla f(\mathbf{x}_k) + \beta_k \mathbf{d}_{k-1}.$$

All coefficients $\beta_{kj} = 0$ for $j < k - 1$.

Full Gram-Schmidt

β_{10}			
β_{20}	β_{21}		
β_{30}	β_{31}	β_{32}	
β_{40}	β_{41}	β_{42}	β_{43}

$O(k^2)$ terms

collapse!
→

CG Recurrence

β_1			
0	β_2		
0	0	β_3	
0	0	0	β_4

Only k terms total!

You get the full Gram-Schmidt orthogonalization for free.

Same cost per step as steepest descent, but convergence of conjugate directions.

Key Lemma: Mutual Orthogonality of Gradients

Lemma

With exact line search and $\mathbf{A}$ -conjugate directions:

$$\nabla f(\mathbf{x}_i)^\top \nabla f(\mathbf{x}_j) = 0 \quad \text{for } i \neq j.$$

Proof. We showed $\nabla f(\mathbf{x}_{k+1})^\top \mathbf{d}_j = 0$ for $j \leq k$. Since each $\mathbf{d}_j$ is a linear combination of $\nabla f(\mathbf{x}_0), \dots, \nabla f(\mathbf{x}_j)$:

$$\text{span}\{\mathbf{d}_0, \dots, \mathbf{d}_k\} = \text{span}\{\nabla f(\mathbf{x}_0), \dots, \nabla f(\mathbf{x}_k)\}.$$

So $\nabla f(\mathbf{x}_{k+1}) \perp \text{span}\{\nabla f(\mathbf{x}_0), \dots, \nabla f(\mathbf{x}_k)\}$. □

The gradients form an orthogonal set! This is the structural property that makes everything collapse.

Intuition: CG generates a new “independent piece of information” at each step. No redundancy.

Proof: Why Gram–Schmidt Collapses

We need: $\beta_{kj} = \frac{\nabla f(\mathbf{x}_k)^\top \mathbf{A} \mathbf{d}_j}{\mathbf{d}_j^\top \mathbf{A} \mathbf{d}_j} = 0$ for $j < k - 1$.

From $\mathbf{r}_k = \mathbf{r}_{k-1} - \alpha_{k-1} \mathbf{A} \mathbf{d}_{k-1}$:

$$\mathbf{A} \mathbf{d}_j = \frac{1}{\alpha_j} (\nabla f(\mathbf{x}_{j+1}) - \nabla f(\mathbf{x}_j)).$$

For $j < k - 1$:

$$\nabla f(\mathbf{x}_k)^\top \mathbf{A} \mathbf{d}_j = \frac{1}{\alpha_j} (\underbrace{\nabla f(\mathbf{x}_k)^\top \nabla f(\mathbf{x}_{j+1})}_{=0} - \underbrace{\nabla f(\mathbf{x}_k)^\top \nabla f(\mathbf{x}_j)}_{=0}) = 0.$$

Both terms vanish by mutual orthogonality! (Since $j + 1 \leq k - 1 < k$.)

For $j = k - 1$:

$$\nabla f(\mathbf{x}_k)^\top \mathbf{A} \mathbf{d}_{k-1} = \frac{1}{\alpha_{k-1}} (\underbrace{\nabla f(\mathbf{x}_k)^\top \nabla f(\mathbf{x}_k)}_{\neq 0} - \underbrace{\nabla f(\mathbf{x}_k)^\top \nabla f(\mathbf{x}_{k-1})}_{=0}) = \frac{\|\nabla f(\mathbf{x}_k)\|^2}{\alpha_{k-1}} \neq 0.$$

Only the most recent term survives.

□

The Fletcher–Reeves Formula

The surviving coefficient:

$$\beta_k = \frac{\nabla f(\mathbf{x}_k)^\top \mathbf{A} \mathbf{d}_{k-1}}{\mathbf{d}_{k-1}^\top \mathbf{A} \mathbf{d}_{k-1}} = \frac{\|\nabla f(\mathbf{x}_k)\|^2 / \alpha_{k-1}}{\mathbf{d}_{k-1}^\top \mathbf{A} \mathbf{d}_{k-1}}.$$

Using $\alpha_{k-1} = \|\nabla f(\mathbf{x}_{k-1})\|^2 / (\mathbf{d}_{k-1}^\top \mathbf{A} \mathbf{d}_{k-1})$ (from the line search + orthogonality):

$$\boxed{\beta_k^{\text{FR}} = \frac{\|\nabla f(\mathbf{x}_k)\|^2}{\|\nabla f(\mathbf{x}_{k-1})\|^2}}$$

Just a ratio of gradient norms. No matrix-vector products needed for β_k .

Entire cost of CG beyond steepest descent: one scalar division per step.

The CG Algorithm

Linear Conjugate Gradient Algorithm

CG for $\min \frac{1}{2} \mathbf{x}^\top \mathbf{A} \mathbf{x} - \mathbf{b}^\top \mathbf{x}$

Initialize: Choose $\mathbf{x}_0$. Set $\mathbf{r}_0 = \mathbf{b} - \mathbf{A}\mathbf{x}_0$, $\mathbf{d}_0 = \mathbf{r}_0$.

For $k = 0, 1, 2, \dots$:

$$\alpha_k = \frac{\mathbf{r}_k^\top \mathbf{r}_k}{\mathbf{d}_k^\top \mathbf{A} \mathbf{d}_k} \quad (\text{step size via exact line search})$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k \quad (\text{update iterate})$$

$$\mathbf{r}_{k+1} = \mathbf{r}_k - \alpha_k \mathbf{A} \mathbf{d}_k \quad (\text{update residual})$$

$$\beta_{k+1} = \frac{\mathbf{r}_{k+1}^\top \mathbf{r}_{k+1}}{\mathbf{r}_k^\top \mathbf{r}_k} \quad (\text{Gram-Schmidt coefficient})$$

$$\mathbf{d}_{k+1} = \mathbf{r}_{k+1} + \beta_{k+1} \mathbf{d}_k \quad (\text{new conjugate direction})$$

Cost per iteration: 1 mat-vec ($\mathbf{A} \mathbf{d}_k$), 2 dots, 3 axpys. **Same as steepest descent.**

Numerical Example: Tracing CG by Hand

$$\mathbf{A} = \begin{pmatrix} 4 & 1 \\ 1 & 3 \end{pmatrix}, \mathbf{b} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \mathbf{x}_0 = \begin{pmatrix} 0 \\ 0 \end{pmatrix}. \quad (\text{Solution: } \mathbf{x}^* = \frac{1}{11}(1, 7)^\top.)$$

Step 0: $\mathbf{r}_0 = (1, 2)^\top$, $\mathbf{d}_0 = (1, 2)^\top$.

$$\mathbf{A}\mathbf{d}_0 = (6, 7)^\top, \quad \alpha_0 = 5/20 = 0.25.$$

$$\mathbf{x}_1 = (0.25, 0.5)^\top, \quad \mathbf{r}_1 = (-0.5, 0.25)^\top.$$

Verify orthogonality: $\mathbf{r}_1^\top \mathbf{r}_0 = (-0.5)(1) + (0.25)(2) = 0 \quad \checkmark$

Step 1: $\beta_1 = (0.25 + 0.0625)/(1 + 4) = 0.0625$.

$$\mathbf{d}_1 = (-0.5, 0.25)^\top + 0.0625(1, 2)^\top = (-0.4375, 0.375)^\top.$$

Verify conjugacy: $\mathbf{d}_1^\top \mathbf{A}\mathbf{d}_0 = (-0.4375, 0.375)(6, 7)^\top = -2.625 + 2.625 = 0 \quad \checkmark$

$$\alpha_1 \approx 0.2078, \quad \mathbf{x}_2 \approx (0.0909, 0.578)^\top.$$

After rounding: $\mathbf{x}_2 = \mathbf{x}^*$ in 2 steps.

CG is Optimal Over Expanding Subspaces

A Deeper View: Krylov Subspaces

At each step, CG doesn't just pick a good direction—it solves an **optimal subproblem**.

Theorem

After k steps of CG, $\mathbf{x}_k$ minimizes f over the affine subspace

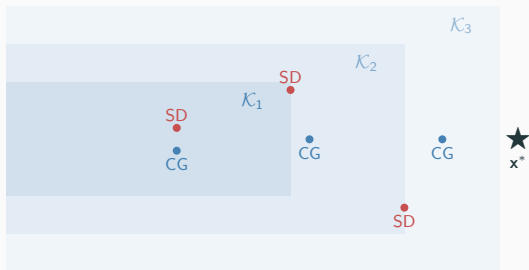
$$\mathbf{x}_0 + \mathcal{K}_k(\mathbf{A}, \mathbf{r}_0),$$

*where $\mathcal{K}_k(\mathbf{A}, \mathbf{r}_0) = \text{span}\{\mathbf{r}_0, \mathbf{A}\mathbf{r}_0, \mathbf{A}^2\mathbf{r}_0, \dots, \mathbf{A}^{k-1}\mathbf{r}_0\}$ is the **Krylov subspace**.*

What is $\mathcal{K}_k$? Start with $\mathbf{r}_0$ (the initial residual). Multiply by $\mathbf{A}$ to get $\mathbf{A}\mathbf{r}_0$ (a “new direction” informed by curvature). Repeat. Each multiplication reveals more about $\mathbf{A}$'s structure.

$$\mathcal{K}_1 = \text{span}\{\mathbf{r}_0\} \rightarrow \mathcal{K}_2 = \text{span}\{\mathbf{r}_0, \mathbf{A}\mathbf{r}_0\} \rightarrow \mathcal{K}_3 = \text{span}\{\mathbf{r}_0, \mathbf{A}\mathbf{r}_0, \mathbf{A}^2\mathbf{r}_0\} \rightarrow \dots$$

Why Krylov Optimality Matters



CG: best point in each subspace

SD: just follows gradient, ignores subspace

Steepest descent picks a point in $\mathcal{K}_k$ but not the *best* one.

CG solves $\min_{\mathbf{x} \in \mathbf{x}_0 + \mathcal{K}_k} f(\mathbf{x})$ *exactly* at each step.

CG is the optimal method that only uses matrix-vector products with **A**.

The Chebyshev Connection (Bonus Intuition)

The convergence bound

$$\|\mathbf{x}_k - \mathbf{x}^*\|_{\mathbf{A}} \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^k \|\mathbf{x}_0 - \mathbf{x}^*\|_{\mathbf{A}}$$

comes from CG minimizing over $\mathcal{K}_k$. This is equivalent to finding the degree- k polynomial p_k with $p_k(0) = 1$ that is smallest on $[\lambda_{\min}, \lambda_{\max}]$:

$$\|\mathbf{e}_k\|_{\mathbf{A}} = \min_{\substack{p_k \text{ deg } k \\ p_k(0)=1}} \max_{\lambda \in [\lambda_{\min}, \lambda_{\max}]} |p_k(\lambda)| \cdot \|\mathbf{e}_0\|_{\mathbf{A}}.$$

The optimal polynomial is a **shifted Chebyshev polynomial**—the polynomial that deviates least from zero on an interval.

The $\sqrt{\kappa}$ arises because Chebyshev polynomials decay exponentially on $[\lambda_{\min}, \lambda_{\max}]$ at rate $(\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)$.

Convergence: What the Rate Really Means

Two Convergence Results

Result 1 — Finite termination: CG terminates in at most n steps.

Result 2 — Convergence rate: $\|\mathbf{x}_k - \mathbf{x}^*\|_{\mathbf{A}} \leq 2 \left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1} \right)^k \|\mathbf{x}_0 - \mathbf{x}^*\|_{\mathbf{A}}.$

These answer different questions:

Finite termination

CG is *correct*:

it will find $\mathbf{x}^*$.

(Worst case over all SPD $\mathbf{A}$.)

Convergence rate

CG is *fast*:

it finds $\mathbf{x}^*$ well before n steps.

(Depends on spectrum, not n .)

When Is the n -Step Result Useless?

If $n = 1,000,000$: “converges in at most 10^6 steps” $\approx$ useless.

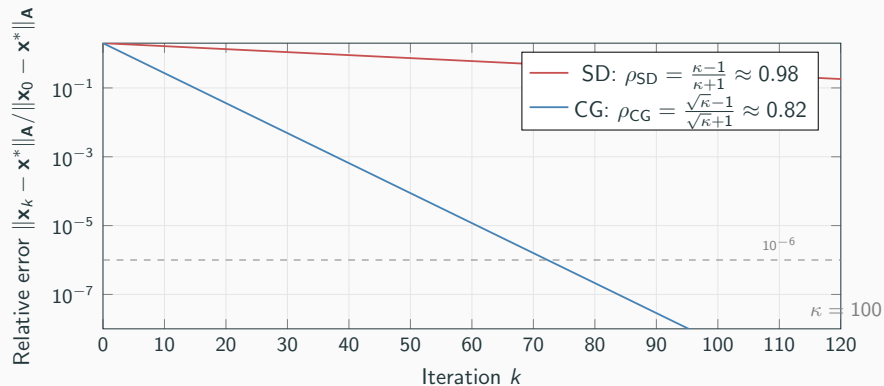
The rate tells you what actually happens:

n	κ	$\rho = \frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}$	Steps to 10^{-6}	Ratio
10^3	10	0.52	21	2.1% of n
10^4	100	0.82	70	0.7% of n
10^5	10^3	0.94	220	0.22% of n
10^6	10^4	0.98	690	0.069% of n

CG depends on $\sqrt{\kappa}$, not n . A million-variable problem with moderate conditioning is easy for CG.

Finite precision: In floating point, the n -step guarantee is lost (rounding degrades orthogonality), but the rate bound remains approximately valid. The rate is more practically relevant.

Convergence Comparison: SD vs. CG



SD needs ~ 700 steps; CG needs ~ 35 steps. CG takes the square root of the condition number.

The Role of the Spectrum

CG convergence depends on the **eigenvalue distribution**, not just κ :

Theorem

If $\mathbf{A}$ has only r distinct eigenvalues, CG terminates in at most r steps.

Spread: 5 distinct



CG needs up to 5 steps

Clustered: 2 groups



CG converges fast (like 2 distinct)

Practical consequence: If you can cluster the eigenvalues of $\mathbf{A}$, CG converges in far fewer steps. This motivates **preconditioning**.

Why Clustering Helps: CG as Polynomial Approximation

After k steps, the CG error satisfies:

$$\|\mathbf{x}_k - \mathbf{x}^*\|_{\mathbf{A}} = \min_{\substack{p_k \text{ deg } k \\ p_k(0)=1}} \|p_k(\mathbf{A}) \mathbf{e}_0\|_{\mathbf{A}}$$

where $\mathbf{e}_0 = \mathbf{x}_0 - \mathbf{x}^*$.

What does this mean? Diagonalize $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$. Then $p_k(\mathbf{A}) = \mathbf{Q} p_k(\mathbf{\Lambda}) \mathbf{Q}^\top$, and:

$$\|\mathbf{x}_k - \mathbf{x}^*\|_{\mathbf{A}}^2 = \sum_{i=1}^n \lambda_i [p_k(\lambda_i)]^2 \hat{\mathbf{e}}_{0,i}^2$$

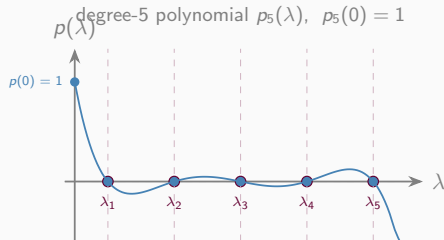
where $\hat{\mathbf{e}}_0 = \mathbf{Q}^\top \mathbf{e}_0$.

CG finds the degree- k polynomial that is smallest at the eigenvalues of $\mathbf{A}$.

A polynomial of degree k has k roots to “spend.” Each root placed near an eigenvalue suppresses that component of the error.

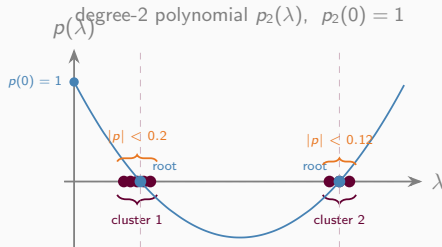
Roots vs. Eigenvalues: The Key Picture

5 Spread Eigenvalues



5 roots placed at 5 eigenvalues
 $\Rightarrow$ need $k \geq 5$ CG steps

2 Clusters of Eigenvalues



Only 2 roots, one per cluster
 $\Rightarrow k = 2$ CG steps nearly suffice

CG “spends” one iteration per cluster. A root near a cluster center makes $|p(\lambda_i)|$ small for all λ_i in that cluster.

A Concrete Example

Two systems, both $n = 1000$, both $\kappa = 100$:

System A: Spread spectrum

990 eigenvalues uniformly on $[1, 100]$.



No clustering structure.

CG rate: $\left(\frac{\sqrt{100}-1}{\sqrt{100}+1}\right)^k = 0.82^k$.

~ 70 steps to 10^{-6} .

System B: Clustered spectrum

990 eigenvalues in $[0.99, 1.01]$,

10 eigenvalues in $[99, 101]$.



2 clusters. CG “finds” both clusters in ~ 2 iterations.

$\sim 5-10$ steps to 10^{-6} .

Same κ , same n . Completely different CG performance. The spectrum—not just κ or n —is what matters.

Worked Example: Setup

Consider three variants of a 6-variable quadratic. In all cases, $\mathbf{x}_0 = (1, 2, 3, 4, 5, 6)^\top$ and $\mathbf{x}^* = \mathbf{0}$.

Case A — Two distinct eigenvalues:

$$f(\mathbf{x}) = x_1^2 + x_2^2 + x_3^2 + 10(x_4^2 + x_5^2 + x_6^2)$$

$$\mathbf{A} = \text{diag}(\underbrace{2, 2, 2}_{\lambda=2}, \underbrace{20, 20, 20}_{\lambda=20}). \quad \text{Only 2 distinct eigenvalues, } \kappa = 10.$$

Case B — Wide clusters (10% perturbation):

$$f(\mathbf{x}) = 0.9x_1^2 + x_2^2 + 1.1x_3^2 + 9x_4^2 + 10x_5^2 + 11x_6^2$$

$$\mathbf{A} = \text{diag}(\underbrace{1.8, 2.0, 2.2}_{\text{cluster} \approx 2}, \underbrace{18, 20, 22}_{\text{cluster} \approx 20}). \quad 6 \text{ distinct eigenvalues, } \kappa \approx 12.$$

Case B' — Tight clusters (1% perturbation):

$$f(\mathbf{x}) = 0.99x_1^2 + x_2^2 + 1.01x_3^2 + 9.9x_4^2 + 10x_5^2 + 10.1x_6^2$$

$$\mathbf{A} = \text{diag}(\underbrace{1.98, 2.0, 2.02}_{\text{cluster} \approx 2}, \underbrace{19.8, 20.0, 20.2}_{\text{cluster} \approx 20}). \quad 6 \text{ distinct, } \kappa \approx 10.$$

Case A: CG Optimizes One Eigenspace Per Step

$\mathbf{A} = \text{diag}(2, 2, 2, 20, 20, 20)$. Track each component of $\mathbf{x}_k$:

k	$\lambda = 2$ components			$\lambda = 20$ components			$\ \mathbf{e}_k\ _{\mathbf{A}}$
	$ x_1 $	$ x_2 $	$ x_3 $	$ x_4 $	$ x_5 $	$ x_6 $	
0	1.000	2.000	3.000	4.000	5.000	6.000	3.96×10^1
1	0.900	1.800	2.700	0.007	0.008	0.010	4.76×10^0
2	0.000	0.000	0.000	0.000	0.000	0.000	3×10^{-14}

What happened:

- **Step 1:** CG nearly zeroes out the $\lambda = 20$ eigenspace (the “steep” directions).
The $\lambda = 2$ components barely move.
- **Step 2:** CG zeroes out the $\lambda = 2$ eigenspace.

Your intuition is exactly right: CG handles one eigenspace per step. With 2 distinct eigenvalues, 2 steps suffice.

Case B: Wide Clusters Require Extra Steps

$\mathbf{A} = \text{diag}(1.8, 2.0, 2.2, 18, 20, 22)$. Within-cluster $\kappa_{\text{within}} = 2.2/1.8 = 1.22$ (22% spread).

k	$\lambda \approx 2$ cluster			$\lambda \approx 20$ cluster			$\ \mathbf{e}_k\ _{\mathbf{A}}$
	$ x_1 $	$ x_2 $	$ x_3 $	$ x_4 $	$ x_5 $	$ x_6 $	
0	1.000	2.000	3.000	4.000	5.000	6.000	4.01×10^1
1	0.913	1.807	2.681	0.524	0.173	0.372	5.70×10^0
2	0.807	1.572	2.299	0.115	0.093	0.098	4.28×10^0
3	0.201	0.257	0.177	0.182	0.210	0.065	1.36×10^0
4	0.115	0.085	0.076	0.001	0.001	0.000	2.25×10^{-1}
5	0.007	0.006	0.002	0.000	0.000	0.000	1.35×10^{-2}
6	0.000	0.000	0.000	0.000	0.000	0.000	3.9×10^{-14}

What happened:

- **Steps 1–2:** CG reduces the $\lambda \approx 20$ cluster ($4,5,6 \rightarrow 0.1$) but *not to zero*—because $\lambda = 18, 20, 22$ are distinct.
- **Steps 3–6:** CG resolves the within-cluster spread.

The 22% within-cluster spread costs 4 extra steps beyond the 2-step “ideal.”

Why Doesn't CG Just Handle Each Group Separately?

Your intuition: "CG should zero out x_4, x_5, x_6 first (the steep directions), then x_1, x_2, x_3 . That's ~ 2 steps."

This is exactly what happens in Case A! Look at step 1:

Case A, step 1: $(x_4, x_5, x_6): (4, 5, 6) \rightarrow (0.007, 0.008, 0.010)$ ✓ zeroed

Case B, step 1: $(x_4, x_5, x_6): (4, 5, 6) \rightarrow (0.524, 0.173, 0.372)$ not zeroed

Why does Case B fail to zero out the group?

In Case A, all three "big" eigenvalues are *identical* ($\lambda = 20, 20, 20$). The CG polynomial places a root at $\lambda = 20$:

$$p(\lambda) = 1 - \lambda/20 \quad \Rightarrow \quad p(20) = 0, \quad p(20) = 0, \quad p(20) = 0 \quad \checkmark$$

In Case B, the eigenvalues are 18, 20, 22. A root at $\lambda = 20$ gives:

$$p(\lambda) = 1 - \lambda/20 \quad \Rightarrow \quad p(18) = 0.1, \quad p(20) = 0, \quad p(22) = -0.1 \quad \text{residual!}$$

The 10% residual at $\lambda = 18$ and $\lambda = 22$ means those components aren't killed—they need additional CG steps to resolve.

Bottom line: One polynomial root can zero out one *eigenvalue*, not one *cluster*—unless the cluster is so tight that $|p(\lambda_i)| \approx 0$ for all λ_i near the root. This is why within-cluster spread matters.

Case B': Tight Clusters $\approx$ Exact Case

$\mathbf{A} = \text{diag}(1.98, 2.0, 2.02, 19.8, 20.0, 20.2)$. Within-cluster $\kappa_{\text{within}} = 1.02$ (2% spread).

k	$\lambda \approx 2$ cluster			$\lambda \approx 20$ cluster			$\ \mathbf{e}_k\ _{\mathbf{A}}$
	$ x_1 $	$ x_2 $	$ x_3 $	$ x_4 $	$ x_5 $	$ x_6 $	
0	1.000	2.000	3.000	4.000	5.000	6.000	3.97×10^1
1	0.901	1.800	2.697	0.044	0.005	0.054	4.78×10^0
2	0.243	0.472	0.687	0.323	0.091	0.276	2.30×10^0
3	0.013	0.009	0.010	0.002	0.003	0.001	3.1×10^{-2}
4	0.011	0.008	0.009	0.000	0.001	0.000	2.4×10^{-2}
5	0.000	0.000	0.000	0.000	0.000	0.000	1.4×10^{-4}
6	0.000	0.000	0.000	0.000	0.000	0.000	2.0×10^{-15}

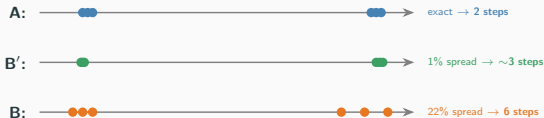
What happened:

- **Step 1:** $\lambda \approx 20$ cluster drops to 0.05—nearly as good as Case A!
- **Step 3:** All components below 10^{-2} . Essentially converged by step 3.
- Steps 4–6 just clean up the last few digits.

2% within-cluster spread $\Rightarrow$ CG behaves nearly like the 2-distinct case.

The Lesson: Within-Cluster Spread Determines Extra Cost

Eigenvalue Spectra



	Within-cluster κ	CG steps	Std. bound	Speedup
A: exact	1.00	2	8	4×
B': 1% perturb	1.02	~3	8	~ 3×
B: 10% perturb	1.22	6	25	4×

The pattern:

- ~2 CG steps to find each cluster center (the “between-cluster” work).
- Additional steps to resolve within-cluster spread (cost $\propto$ within-cluster κ).
- **Preconditioning** should aim to make within-cluster $\kappa \approx 1$.

Where the $\sqrt{\kappa}$ Rate Comes From

When eigenvalues are spread on $[\lambda_{\min}, \lambda_{\max}]$ with no clustering structure, the optimal polynomial is a **shifted Chebyshev polynomial**:

$$p_k(\lambda) = \frac{T_k\left(\frac{\lambda_{\max} + \lambda_{\min} - 2\lambda}{\lambda_{\max} - \lambda_{\min}}\right)}{T_k\left(\frac{\lambda_{\max} + \lambda_{\min}}{\lambda_{\max} - \lambda_{\min}}\right)}$$

Chebyshev polynomials T_k grow exponentially outside $[-1, 1]$:

$$T_k(x) \approx \frac{1}{2} \left(x + \sqrt{x^2 - 1} \right)^k \quad \text{for } |x| > 1.$$

Substituting and simplifying:

$$\max_{\lambda \in [\lambda_{\min}, \lambda_{\max}]} |p_k(\lambda)| \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^k.$$

The $\sqrt{\kappa}$ is the Chebyshev polynomial's exponential decay rate on $[\lambda_{\min}, \lambda_{\max}]$.

With clustering, you beat this bound because the polynomial only needs to be small at the cluster locations, not on the entire interval.

Preconditioning: Making CG Practical

The Preconditioning Idea

Problem: CG converges slowly when $\kappa(\mathbf{A})$ is large.

Idea: Find a matrix $\mathbf{M} \approx \mathbf{A}$ that is cheap to invert. Then solve the *transformed* system $\mathbf{M}^{-1}\mathbf{A}\mathbf{x} = \mathbf{M}^{-1}\mathbf{b}$ instead.

If $\mathbf{M} \approx \mathbf{A}$, then $\mathbf{M}^{-1}\mathbf{A} \approx \mathbf{I}$, so $\kappa(\mathbf{M}^{-1}\mathbf{A}) \ll \kappa(\mathbf{A})$.

Without Preconditioner



$$\kappa(\mathbf{A}) = 1000$$

CG needs ~220 steps



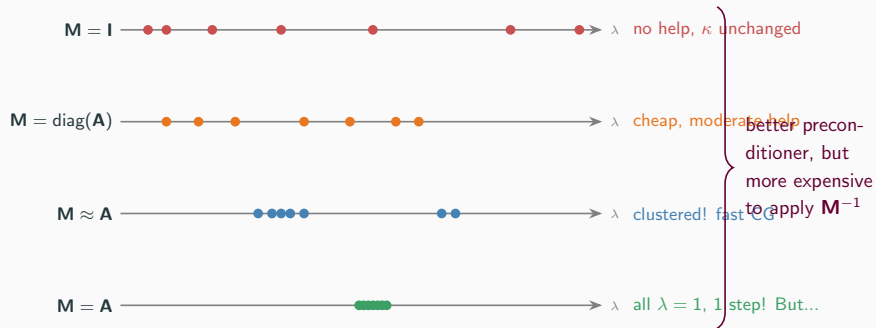
With Preconditioner



$$\kappa(\mathbf{M}^{-1}\mathbf{A}) = 10$$

CG needs ~21 steps

The Tension in Choosing M



The trade-off: More accurate $M \approx A \Rightarrow$ fewer CG iterations, but each iteration includes a M^{-1} solve.

Good preconditioners: incomplete Cholesky, multigrid, domain decomp.

Preconditioned CG Algorithm

PCG for $Ax = b$ with preconditioner M

Initialize: $r_0 = b - Ax_0$, solve $Mz_0 = r_0$, $d_0 = z_0$.

For $k = 0, 1, 2, \dots$:

$$\alpha_k = \frac{r_k^\top z_k}{d_k^\top A d_k}$$

$$x_{k+1} = x_k + \alpha_k d_k$$

$$r_{k+1} = r_k - \alpha_k A d_k$$

$$\text{Solve } Mz_{k+1} = r_{k+1}$$

$$\beta_{k+1} = \frac{r_{k+1}^\top z_{k+1}}{r_k^\top z_k}$$

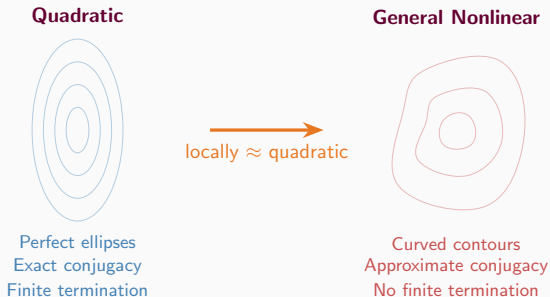
$$d_{k+1} = z_{k+1} + \beta_{k+1} d_k$$

Only change: replace r with $z = M^{-1}r$ in the formulas. Add one M^{-1} solve per iteration.

Nonlinear CG

From Quadratics to General Functions

For general $f(\mathbf{x})$, the quadratic structure breaks:



Approach: Keep $\mathbf{d}_k = -\nabla f(\mathbf{x}_k) + \beta_k \mathbf{d}_{k-1}$ but choose β_k to work without exact quadratic structure.

But why should this work? Everything we derived depended on the quadratic structure.

What Breaks: Everything That Made CG Special

In the quadratic case, three properties made CG work:

Quadratic

Fixed $\mathbf{A}$ -inner product
 $\langle \mathbf{u}, \mathbf{v} \rangle_{\mathbf{A}} = \mathbf{u}^{\top} \mathbf{A} \mathbf{v}$

Gradients mutually $\perp$
 $\nabla f(\mathbf{x}_i)^{\top} \nabla f(\mathbf{x}_j) = 0$

Gram-Schmidt collapses
to one-term recurrence

General Nonlinear

Hessian changes
at every $\mathbf{x}_k$
No fixed inner product

Gradients **not** orthogonal
 $\nabla f(\mathbf{x}_i)^{\top} \nabla f(\mathbf{x}_j) \neq 0$

Short recurrence is
not justified by algebra

The *derivation* is gone. What remains is the *structure*: $\mathbf{d}_k = -\nabla f(\mathbf{x}_k) + \beta_k \mathbf{d}_{k-1}$.

“Permanent Progress” Is Gone

On a quadratic, “permanent progress” meant:

$$\nabla f(\mathbf{x}_{k+1})^\top \mathbf{d}_j = 0 \quad \forall j \leq k.$$

The gradient has *no component* along any previous direction. Ever. This was guaranteed by **A**-conjugacy with a *fixed A*.

On a general function, the curvature at $\mathbf{x}_k$ is different from the curvature at $\mathbf{x}_0$:

Quadratic



Same ellipses everywhere
Curvature is **constant**

General Nonlinear



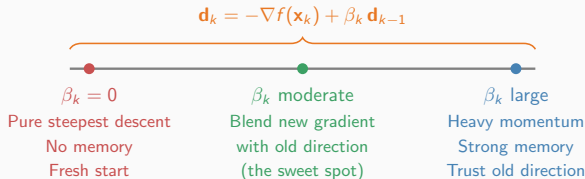
Contour shapes **change**
“Conjugate to what?”

A direction that was “conjugate” to the curvature at $\mathbf{x}_0$ has no special relationship to the curvature at $\mathbf{x}_{10}$.

What β_k Really Means in the Nonlinear Setting

On a quadratic: β_k is the Gram–Schmidt coefficient—the $\mathbf{A}$ -projection that makes $\mathbf{d}_k$ conjugate to $\mathbf{d}_{k-1}$. Precise geometric meaning.

On a general function: β_k controls how much memory to retain:



The different β_k formulas are different answers to: “How should I set this dial given only gradient information?”

They’re not computing an $\mathbf{A}$ -projection (there’s no fixed $\mathbf{A}$). They’re *estimating* the right amount of memory.

PRP Uses Curvature; FR Doesn't

Why does PRP work better? Look at what's inside:

$$\beta_k^{\text{PRP}} = \frac{\nabla f(\mathbf{x}_k)^\top \overbrace{(\nabla f(\mathbf{x}_k) - \nabla f(\mathbf{x}_{k-1}))}^{y_k = \text{gradient change}}}{\|\nabla f(\mathbf{x}_{k-1})\|^2}$$

By Taylor's theorem: $\nabla f(\mathbf{x}_k) - \nabla f(\mathbf{x}_{k-1}) \approx \nabla^2 f(\mathbf{x}_{k-1}) \mathbf{s}_{k-1}$.

The gradient change is a free Hessian-vector product!

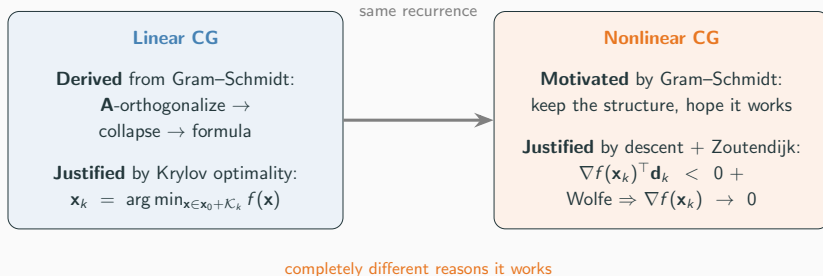
PRP is implicitly using curvature information—it's doing a crude, one-step version of what BFGS does systematically with $(\mathbf{s}_k, \mathbf{y}_k)$ pairs.

Compare FR: $\beta_k^{\text{FR}} = \frac{\|\nabla f(\mathbf{x}_k)\|^2}{\|\nabla f(\mathbf{x}_{k-1})\|^2}$ — only gradient *magnitudes*. No direction information, no curvature.

This is why:

- PRP adapts to the local geometry ($\beta \rightarrow 0$ when curvature flattens)
- FR is blind to geometry (same β whether curvature changed or not)

The Conceptual Jump: Derivation vs. Justification



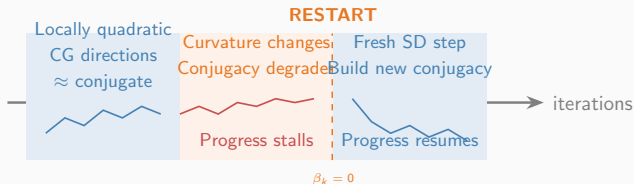
The recurrence $\mathbf{d}_k = -\nabla f(\mathbf{x}_k) + \beta_k \mathbf{d}_{k-1}$ is identical. The reason it works is completely different.

Linear: algebraic miracle $\Rightarrow$ optimal over expanding subspaces.

Nonlinear: local quadratic approximation + line search safety net.

Why Restarts Exist: An Admission

Restarts (set $\beta_k = 0$ periodically) are an admission that the Gram–Schmidt analogy has a **limited shelf life** on nonlinear functions.



Common strategies:

- Restart every n steps (Powell, 1977)
- Restart when $|\nabla f(\mathbf{x}_k)^\top \nabla f(\mathbf{x}_{k-1})| > 0.2 \|\nabla f(\mathbf{x}_k)\|^2$ (Powell, 1984):
“orthogonality is too degraded”
- Use PRP — which restarts *automatically* via $\beta \approx 0$ when curvature flattens

The Bet Nonlinear CG is Making

Every step of nonlinear CG is a bet:

“The curvature hasn’t changed *too much* since the last step, so a direction that was good for the *previous* local quadratic is still partially good for the *current* one.”

- **When the bet pays off** (smooth, mildly nonlinear f): CG converges much faster than steepest descent, approaching its quadratic rate.
- **When the bet fails** (rapidly changing curvature, saddle points, sharp turns): the old direction is misleading. PRP detects this; FR doesn’t.
- **Restarts** are the safety net: when the bet has been failing too long, cut losses and start fresh.

BFGS makes the same bet but hedges more carefully — it accumulates curvature information from *all* previous steps, not just the last one.

The Descent Property: Why CG Gives Descent Directions

For CG to work with line search, we need $\mathbf{d}_k$ to be a **descent direction**:

$$\nabla f(\mathbf{x}_k)^\top \mathbf{d}_k < 0.$$

On a quadratic:

$$\begin{aligned}\nabla f(\mathbf{x}_k)^\top \mathbf{d}_k &= \nabla f(\mathbf{x}_k)^\top (-\nabla f(\mathbf{x}_k) + \beta_k \mathbf{d}_{k-1}) = -\|\nabla f(\mathbf{x}_k)\|^2 + \beta_k \underbrace{\nabla f(\mathbf{x}_k)^\top \mathbf{d}_{k-1}}_{=0 \text{ (exact LS)}} \\ &= -\|\nabla f(\mathbf{x}_k)\|^2 < 0. \quad \checkmark\end{aligned}$$

On a general function with inexact line search:

$\nabla f(\mathbf{x}_k)^\top \mathbf{d}_{k-1} \neq 0$ in general. The descent condition becomes:

$$\nabla f(\mathbf{x}_k)^\top \mathbf{d}_k = -\|\nabla f(\mathbf{x}_k)\|^2 + \beta_k \nabla f(\mathbf{x}_k)^\top \mathbf{d}_{k-1}.$$

This is negative as long as $|\beta_k \nabla f(\mathbf{x}_k)^\top \mathbf{d}_{k-1}| < \|\nabla f(\mathbf{x}_k)\|^2$.

This is why the Wolfe line search parameter c_2 matters for CG.

Why CG Needs $c_2 < \frac{1}{2}$ in the Wolfe Conditions

Recall the **strong Wolfe conditions**:

$$f(\mathbf{x}_k + \alpha_k \mathbf{d}_k) \leq f(\mathbf{x}_k) + c_1 \alpha_k \nabla f(\mathbf{x}_k)^\top \mathbf{d}_k \quad (\text{sufficient decrease})$$

$$|\nabla f(\mathbf{x}_{k+1})^\top \mathbf{d}_k| \leq c_2 |\nabla f(\mathbf{x}_k)^\top \mathbf{d}_k| \quad (\text{curvature condition})$$

The curvature condition gives us $|\nabla f(\mathbf{x}_{k+1})^\top \mathbf{d}_k| \leq c_2 |\nabla f(\mathbf{x}_k)^\top \mathbf{d}_k|$.

For FR: $\beta_{k+1}^{\text{FR}} = \|\nabla f(\mathbf{x}_{k+1})\|^2 / \|\nabla f(\mathbf{x}_k)\|^2$. One can show that if $c_2 < \frac{1}{2}$:

$$\nabla f(\mathbf{x}_{k+1})^\top \mathbf{d}_{k+1} \leq - \left(1 - \frac{1}{1 - c_2}\right) \|\nabla f(\mathbf{x}_{k+1})\|^2 < 0.$$

With $c_2 \geq \frac{1}{2}$, CG can produce ascent directions!

Compare: BFGS works fine with $c_2 = 0.9$. CG is more delicate because it stores less curvature information.

Recommendation: Use $c_1 = 10^{-4}$, $c_2 = 0.1$ for CG. (Nocedal & Wright suggest $c_2 \leq 0.4$.)

The β_k Formulas

On a quadratic with exact line search, these are all equivalent. On general functions, they differ. Let $\mathbf{y}_k = \nabla f(\mathbf{x}_k) - \nabla f(\mathbf{x}_{k-1})$ (the gradient change):

Name	β_k	Key property
Fletcher-Reeves	$\frac{\ \nabla f(\mathbf{x}_k)\ ^2}{\ \nabla f(\mathbf{x}_{k-1})\ ^2}$	Safest theory; can stagnate
Polak-Ribière-Polyak	$\frac{\nabla f(\mathbf{x}_k)^\top \mathbf{y}_k}{\ \nabla f(\mathbf{x}_{k-1})\ ^2}$	Auto-restart; best practice
Hestenes-Stiefel	$\frac{\nabla f(\mathbf{x}_k)^\top \mathbf{y}_k}{\mathbf{d}_{k-1}^\top \mathbf{y}_k}$	$\equiv$ memoryless BFGS
Dai-Yuan	$\frac{\ \nabla f(\mathbf{x}_k)\ ^2}{\mathbf{d}_{k-1}^\top \mathbf{y}_k}$	Good theory with Wolfe

FR vs. PRP: Why Auto-Restart Matters

Key difference: $\beta_k^{\text{PRP}} = \nabla f(\mathbf{x}_k)^\top \mathbf{y}_k / \|\nabla f(\mathbf{x}_{k-1})\|^2$ where $\mathbf{y}_k = \nabla f(\mathbf{x}_k) - \nabla f(\mathbf{x}_{k-1})$.

When $\nabla f(\mathbf{x}_k) \approx \nabla f(\mathbf{x}_{k-1})$ (the function is nearly flat, or CG is stalling):

Fletcher–Reeves

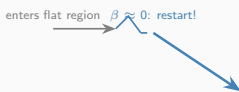
$$\beta_k^{\text{FR}} = \|\nabla f(\mathbf{x}_k)\|^2 / \|\nabla f(\mathbf{x}_{k-1})\|^2 \approx 1$$



Keeps searching a bad direction. Stagnates.

Polak–Ribière–Polyak

$$\beta_k^{\text{PRP}} = \nabla f(\mathbf{x}_k)^\top (\nabla f(\mathbf{x}_k) - \nabla f(\mathbf{x}_{k-1})) / \|\nabla f(\mathbf{x}_{k-1})\|^2 \approx 0$$



Restarts as SD. Makes progress.

PRP detects when conjugacy has degraded and automatically restarts.

FR blindly continues with $\beta \approx 1$, producing poor search directions.

The Math Behind PRP Auto-Restart

When does CG stall? When consecutive iterates are close: $\mathbf{x}_k \approx \mathbf{x}_{k-1}$, so $\nabla f(\mathbf{x}_k) \approx \nabla f(\mathbf{x}_{k-1})$.

What happens to β_k ?

Fletcher–Reeves:

$$\beta_k^{\text{FR}} = \frac{\|\nabla f(\mathbf{x}_k)\|^2}{\|\nabla f(\mathbf{x}_{k-1})\|^2}$$

Since $\nabla f(\mathbf{x}_k) \approx \nabla f(\mathbf{x}_{k-1})$:

$$\beta_k^{\text{FR}} \approx \frac{\|\nabla f(\mathbf{x}_{k-1})\|^2}{\|\nabla f(\mathbf{x}_{k-1})\|^2} = 1.$$

Then:

$$\mathbf{d}_k = -\nabla f(\mathbf{x}_k) + 1 \cdot \mathbf{d}_{k-1}$$

Full weight on the old direction. CG keeps pushing in a direction that isn't making progress.

Polak–Ribière–Polyak:

$$\beta_k^{\text{PRP}} = \frac{\nabla f(\mathbf{x}_k)^\top \underbrace{(\nabla f(\mathbf{x}_k) - \nabla f(\mathbf{x}_{k-1}))}_{\approx \mathbf{0}}}{\|\nabla f(\mathbf{x}_{k-1})\|^2}$$

Since $\nabla f(\mathbf{x}_k) \approx \nabla f(\mathbf{x}_{k-1})$:

$$\beta_k^{\text{PRP}} \approx \frac{\nabla f(\mathbf{x}_k)^\top \cdot \mathbf{0}}{\|\nabla f(\mathbf{x}_{k-1})\|^2} = 0.$$

Then:

$$\mathbf{d}_k = -\nabla f(\mathbf{x}_k) + 0 \cdot \mathbf{d}_{k-1} = -\nabla f(\mathbf{x}_k)$$

Old direction dropped. Fresh steepest descent step.

The mechanism: PRP uses $\mathbf{y}_k = \nabla f(\mathbf{x}_k) - \nabla f(\mathbf{x}_{k-1})$ (the gradient *change*) in the numerator. When progress stalls, $\mathbf{y}_k \approx \mathbf{0}$, so $\beta_k^{\text{PRP}} \approx 0$, which resets the search direction to steepest descent. FR uses $\|\nabla f(\mathbf{x}_k)\|^2$ in the numerator, which stays large even when nothing is changing.

On a quadratic, this distinction vanishes. Since $\nabla f(\mathbf{x}_k)^\top \nabla f(\mathbf{x}_{k-1}) = 0$ (gradient orthogonality), PRP's numerator is $\nabla f(\mathbf{x}_k)^\top (\nabla f(\mathbf{x}_k) - \nabla f(\mathbf{x}_{k-1})) = \|\nabla f(\mathbf{x}_k)\|^2 - 0 = \|\nabla f(\mathbf{x}_k)\|^2$, which is exactly FR. The auto-restart mechanism is invisible because CG never stalls on a quadratic.

Convergence of Nonlinear CG via Zoutendijk

Connection to what you've already seen:

From the Zoutendijk theorem with strong Wolfe conditions:

$$\sum_{k=0}^{\infty} \frac{(\nabla f(\mathbf{x}_k)^\top \mathbf{d}_k)^2}{\|\mathbf{d}_k\|^2} < \infty.$$

Since $\mathbf{d}_k$ is a descent direction (guaranteed by $c_2 < 1/2$), this implies

$$\cos^2 \theta_k \cdot \|\nabla f(\mathbf{x}_k)\|^2 \rightarrow 0 \quad \text{where} \quad \cos \theta_k = \frac{-\nabla f(\mathbf{x}_k)^\top \mathbf{d}_k}{\|\nabla f(\mathbf{x}_k)\| \|\mathbf{d}_k\|}.$$

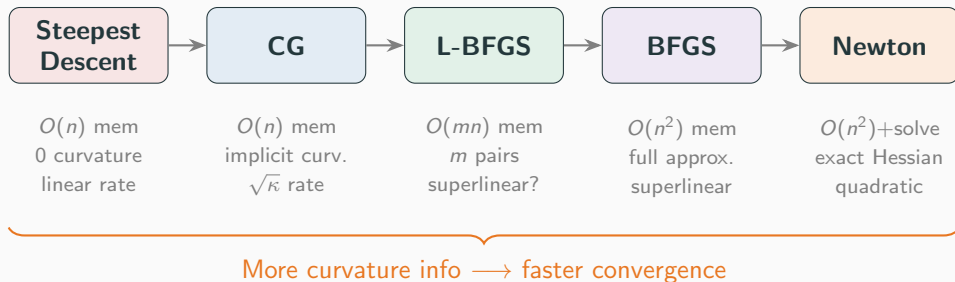
For convergence, we need $\cos \theta_k$ bounded away from zero (directions don't become perpendicular to the gradient).

- **FR**: $\cos \theta_k$ stays bounded $\Rightarrow$ global convergence. But it can be very small $\Rightarrow$ slow.
- **PRP⁺**: $\beta_k = \max(\beta_k^{\text{PRP}}, 0)$ ensures descent; the restart mechanism keeps $\cos \theta_k$ healthy.

Zoutendijk gives the framework; the β_k formula determines the quality.

CG in the Family of Methods

The Memory–Convergence Spectrum



Key insight: CG with Hestenes–Stiefel $\beta_k \equiv$ memoryless BFGS (Shanno '78). CG is the $m = 0$ limit of L-BFGS.

When to Use What

CG when:

- n very large ($> 10K$)
- Memory is scarce
- Good preconditioner exists
- Function mildly nonlinear

L-BFGS when:

- Can afford $O(mn)$ storage
- Highly varying curvature
- Want robustness
- Default for large-scale NLP

BFGS when:

- $n < 1,000$
- Want superlinear convergence
- Can afford $O(n^2)$ memory

Newton when:

- Hessian available & cheap
- Near a solution (local conv.)
- Need quadratic convergence

Case Study: Seismic Full Waveform Inversion

Imaging the Earth's Interior

When an earthquake occurs, seismic waves propagate through the Earth, bouncing off rock layers and structures.

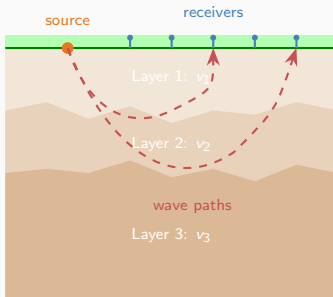
Seismometers on the surface record the vibrations.

The inverse problem: Given recordings at the surface, what does the interior look like?

This is **full waveform inversion (FWI)**—one of the largest-scale optimization problems routinely solved in science.

Applications:

- Oil & gas exploration
- Earthquake hazard assessment



The Wave Equation

Seismic waves are governed by the **acoustic wave equation**:

$$\frac{1}{v(\mathbf{x})^2} \frac{\partial^2 u}{\partial t^2} - \nabla^2 u = s(\mathbf{x}, t)$$

where:

- $u(\mathbf{x}, t)$: pressure wavefield (what we simulate)
- $v(\mathbf{x})$: **seismic velocity** at position $\mathbf{x}$ (what we want to find)
- $s(\mathbf{x}, t)$: source term (known earthquake or explosion)

Discretize on a grid with n cells. The velocity model is a vector $\mathbf{m} \in \mathbb{R}^n$:

$$\mathbf{m} = (v(\mathbf{x}_1), v(\mathbf{x}_2), \dots, v(\mathbf{x}_n))^{\top}.$$

Forward problem: Given $\mathbf{m}$, solve the wave equation $\Rightarrow$ predicted data $\mathbf{d}_s(\mathbf{m})$ at receivers.

Inverse problem: Given observed data $\mathbf{d}_s^{\text{obs}}$, find the velocity model $\mathbf{m}$.

The Optimization Problem

FWI minimizes the **waveform misfit** between observed and predicted data:

$$\min_{\mathbf{m} \in \mathbb{R}^n} f(\mathbf{m}) = \frac{1}{2} \sum_{s=1}^S \|\mathbf{d}_s^{\text{obs}} - \mathbf{d}_s(\mathbf{m})\|^2 + \lambda R(\mathbf{m})$$

- S : number of seismic sources (earthquakes or shots)
- $\mathbf{d}_s(\mathbf{m})$: synthetic seismograms from solving the wave equation with model $\mathbf{m}$
- $R(\mathbf{m})$: regularization (Tikhonov for smooth models, total variation for sharp boundaries)

Problem sizes in practice:

Scale	Grid	n (variables)	Sources S
Near-surface	100×50	$\sim 5,000$	10–50
Exploration (Marmousi)	460×150	$\sim 69,000$	32
Regional (western US)	3D mesh	$\sim 500,000$	~ 200
Global (Earth's mantle)	3D spectral elem.	$\sim 10,000,000$	$\sim 1,500$

Why FWI Is a Gradient-Only Problem

Cost of one function evaluation:

Evaluate $f(\mathbf{m})$ = solve the wave equation S times (one per source).

For the Marmousi benchmark: $S = 32$ forward simulations.

For global imaging (GLAD-M25): $S = 1,480$ forward simulations.

What about second-order methods?

Newton / BFGS

Hessian: $n \times n$ matrix
 $69,000^2 \approx 4.8 \times 10^9$ entries
 ~ 36 GB storage
Impossible.

Hessian-vector product

Requires one additional
wave equation solve
per product.
Expensive.

L-BFGS ($m = 5$)

Store 10 vectors of length n
 ~ 5 MB
Two-loop recursion: $O(mn)$
Feasible!

The gradient is all you can afford. CG and L-BFGS are the natural methods.

Computing the Gradient: The Adjoint-State Method

The gradient $\nabla f(\mathbf{m})$ can be computed **without forming the Jacobian** using the adjoint method.

Step 1: Forward simulation. Solve the wave equation forward in time:

$$\frac{1}{v(\mathbf{x})^2} \frac{\partial^2 u_s}{\partial t^2} - \nabla^2 u_s = s(\mathbf{x}, t) \quad \Rightarrow \quad u_s(\mathbf{x}, t)$$

Step 2: Adjoint simulation. Solve the wave equation *backward* in time with the data residual as the source:

$$\frac{1}{v(\mathbf{x})^2} \frac{\partial^2 u_s^\dagger}{\partial t^2} - \nabla^2 u_s^\dagger = \mathbf{d}_s^{\text{obs}} - \mathbf{d}_s(\mathbf{m}) \quad \Rightarrow \quad u_s^\dagger(\mathbf{x}, t)$$

Step 3: Cross-correlate. The gradient at grid point i is:

$$\frac{\partial f}{\partial m_i} = -\frac{2}{v(\mathbf{x}_i)^3} \sum_{s=1}^S \int_0^T \frac{\partial u_s}{\partial t}(\mathbf{x}_i, t) \frac{\partial u_s^\dagger}{\partial t}(\mathbf{x}_i, t) dt$$

Total cost: $2S$ wave equation solves (one forward + one adjoint per source).

This is a matrix-free gradient. No Jacobian, no Hessian—just PDE solves.

Applying CG to FWI

With the adjoint gradient, nonlinear CG proceeds as:

Initialize: Starting velocity model $\mathbf{m}_0$ (e.g., smooth 1D profile), $\mathbf{d}_0 = -\nabla f(\mathbf{m}_0)$.

For $k = 0, 1, 2, \dots$:

1. **Gradient:** Compute $\nabla f(\mathbf{x}_k) = \nabla f(\mathbf{m}_k)$ via adjoint method (2S PDE solves)
2. **Precondition:** $\hat{\mathbf{g}}_k = \mathbf{P}_k \nabla f(\mathbf{x}_k)$ (diagonal scaling for depth/illumination)
3. **Search direction:** With $\mathbf{y}_k = \nabla f(\mathbf{x}_k) - \nabla f(\mathbf{x}_{k-1})$:

$$\mathbf{d}_k = -\hat{\mathbf{g}}_k + \beta_k \mathbf{d}_{k-1}, \quad \beta_k = \frac{\hat{\mathbf{g}}_k^\top \mathbf{y}_k}{\hat{\mathbf{g}}_{k-1}^\top \nabla f(\mathbf{x}_{k-1})} \quad (\text{preconditioned PRP})$$

4. **Line search:** Find α_k satisfying Wolfe conditions (~ 3 – 4 extra f evaluations = 3 – $4 \times S$ PDE solves)
5. **Update:** $\mathbf{m}_{k+1} = \mathbf{m}_k + \alpha_k \mathbf{d}_k$

Each CG iteration costs $\sim 5S$ – $6S$ wave equation solves.

For Marmousi ($S = 32$): ~ 160 – 190 PDE solves per iteration.

Applying L-BFGS to FWI

L-BFGS replaces step 3 with:

3'. **Search direction via two-loop recursion:**

$$\mathbf{d}_k = -\mathbf{H}_k \nabla f(\mathbf{x}_k)$$

where $\mathbf{H}_k$ is the L-BFGS approximation to $(\nabla^2 f)^{-1}$, built from the last m pairs $\{(\mathbf{s}_j, \mathbf{y}_j)\}_{j=k-m}^{k-1}$ with $\mathbf{s}_j = \mathbf{m}_{j+1} - \mathbf{m}_j$, $\mathbf{y}_j = \nabla f(\mathbf{x}_{j+1}) - \nabla f(\mathbf{x}_j)$.

Key advantages over CG for FWI:

- **Simpler line search:** Backtracking suffices (~ 1.2 function evaluations vs. ~ 3.5 for CG's bracketing)
- **Better curvature model:** $m = 5$ correction pairs capture inter-layer tradeoffs that CG misses
- **Step length:** L-BFGS has a natural initial step length from the quadratic model; CG does not

Each L-BFGS iteration costs $\sim 2S-3S$ wave equation solves.

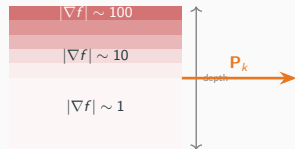
For Marmousi ($S = 32$): $\sim 65-95$ PDE solves per iteration.

Savings: $\sim 50\%$ fewer PDE solves per iteration vs. CG, primarily from the cheaper line search.

Why Preconditioning Is Essential for FWI

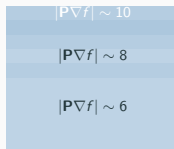
The FWI gradient is dominated by **near-surface** contributions:

Unpreconditioned Gradient



CG refines shallow model,
deep structure barely moves

Preconditioned Gradient



Balanced updates at
all depths

Preconditioner: $P_{ii} = 1 / \sum_{s=1}^S \int_0^T |u_s(\mathbf{x}_i, t)|^2 dt$ (inverse source illumination).

This compensates for geometric spreading: deep grid points get amplified, shallow ones dampened.

Without preconditioning: effective $\kappa \sim 10^4$ – 10^6 . **With:** $\kappa \sim 10$ – 100 .

The Marmousi Benchmark: A Head-to-Head Comparison

The Marmousi model: $9,200 \times 3,000$ m, $n \approx 69,000$ unknowns, 32 sources, 500 receivers. Complex geology with faults, anticlines, and velocity inversions.

Modrak & Tromp (2016) ran **hundreds of inversions**, varying one aspect at a time:

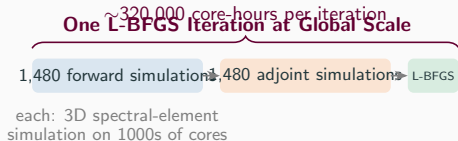
	Nonlinear CG (PRP)	L-BFGS ($m = 5$)
Gradient evaluations to converge	$\sim 80\text{--}100$	$\sim 50\text{--}65$
Line search evals per step	~ 3.5 (bracketing)	~ 1.2 (backtracking)
PDE solves per iteration	$\sim 5S\text{--}6S$	$\sim 2S\text{--}3S$
Extra storage	2 vectors (~ 1 MB)	10 vectors (~ 5 MB)
Restarts needed	$\sim 1\%$ of iterations	$\sim 1\%$ of iterations
Wolfe c_2	< 0.5 required	0.9 works fine
Total computational savings	baseline	30–50% cheaper

Storing 10 extra vectors (~ 5 MB) saves 30–50% of the total computation.

This is the memory–convergence tradeoff made concrete.

Global Scale: Imaging the Earth's Mantle

GLAD-M25 (Lei et al., 2020): A global model of the Earth's mantle.



- $n \sim 10,000,000$ parameters (3D unstructured mesh)
- $S = 1,480$ earthquakes worldwide
- L-BFGS with $m = 5$: converged in ~ 25 iterations
- **Total cost: ~ 8 million core-hours**

If CG needed 50% more iterations, that would be ~ 4 million additional core-hours.

The choice of CG vs. L-BFGS is not academic—it determines whether the computation fits in the supercomputer allocation.

What FWI Teaches About CG and L-BFGS

1. Matrix-free = CG/L-BFGS territory

When you can only compute ∇f (not $\nabla^2 f$), CG and L-BFGS are the methods of choice.

2. Preconditioning is everything

Reduces effective κ from 10^6 to 10^2 .
Connects directly to eigenvalue clustering.

3. Memory–convergence tradeoff

5 MB of extra storage saves millions of core-hours. L-BFGS wins by storing curvature.

4. Line search costs dominate

Each function evaluation = 5 PDE solves.
L-BFGS backtracking (1.2 evals) vs.
CG bracketing (3.5 evals) is decisive.

CG was the standard for 20+ years. L-BFGS displaced it not because CG was wrong, but because storing 10 extra vectors buys enough curvature to meaningfully accelerate convergence.

At this scale, “meaningful” means millions of core-hours.

Summary

Summary

1. **Steepest descent zigzags:** steps undo each other's progress.
2. **Conjugate directions:** **A**-orthogonality $\Rightarrow$ permanent progress, finite termination in n steps.
3. **Building them = A-Gram-Schmidt.** But full GS is $O(n^2)$.
4. **The miracle:** gradient orthogonality $\Rightarrow$ Gram-Schmidt collapses to a **one-term recurrence**. CG costs the same as SD per step.
5. **CG is optimal** over expanding Krylov subspaces. Rate $(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1})^k$ depends on spectrum, not n .
6. **Preconditioning** clusters eigenvalues $\Rightarrow$ fast CG. Essential in practice.
7. **Nonlinear CG:** PRP has auto-restart; needs $c_2 < 1/2$ in Wolfe. Convergence via Zoutendijk.
8. **Case study:** Seismic FWI uses CG/L-BFGS on problems with $n = 10^4$ – 10^7 variables. L-BFGS saves 30–50% over CG by storing 10 extra vectors.

Further Reading

- **Shewchuk** (1994), “An Introduction to the Conjugate Gradient Method Without the Agonizing Pain” — outstanding, freely available
- **Nocedal & Wright**, *Numerical Optimization*, Ch. 5 — standard reference, clean proofs
- **Hager & Zhang** (2006), “A Survey of Nonlinear Conjugate Gradient Methods,” *Pacific J. Optim.* — comprehensive review with CG_DESCENT
- **Andrei** (2020), *Nonlinear CG Methods for Unconstrained Optimization*, Springer — definitive monograph
- **Trefethen & Bau** (1997), *Numerical Linear Algebra*, Lecture 38 — the Krylov/Chebyshev perspective
- **Nazareth** (1979), *SIAM J. Numer. Anal.* — the CG $\leftrightarrow$ BFGS connection